

中山大学 计算机学院本科生实验报告

(2025/2026学年)

课程名称	并行程序设计与算法 (实验)	批改人	
实验	Lab 3 - Pthreads 并行矩阵乘法与数组求和	专业 (方向)	计算机科学与技术 (人工智能)
学号	23336103	姓名	雷颜玮
Email	leiyanwei2005@163.com	完成日期	2026年4月20日

1. 实验要求与目的

使用 Pthreads 实现两种基础的并行计算任务：并行矩阵乘法与并行数组求和，并测试其在不同规模数据与不同线程数下的并行性能（时间、加速比、并行效率）。理解共享内存模型下的线程竞争原理与基于数据块划分的计算调度。

2. 并行矩阵乘法 (Pthreads)

2.1 算法设计与并行逻辑

由于矩阵乘法的结果矩阵 C 中的每个元素计算相互独立，不存在数据依赖与竞争，采用**块数据划分 (Block Data Distribution)**的思路，将 m 行计算任务近似均分给 num_threads 个线程。每个工作线程仅计算分配给自己的这一段行区间的元素，并在计算完成后自动退出。主线程调用 pthread_join 阻塞等待所有子级线程完成汇合。

2.2 核心代码实现

在主线程中划分算力资源并拉起子线程，子线程执行标准的矩阵计算逻辑。

```
// 1. 工作线程：仅计算被分配的那些行 [start_row, end_row)
void* gemm_worker(void* arg) {
    ThreadData* data = (ThreadData*)arg;
    for (int i = data->start_row; i < data->end_row; ++i) {
        for (int k = 0; k < data->k; ++k) {
            double temp = data->A[i * data->k + k];
            for (int j = 0; j < data->n; ++j) {
                data->C[i * data->n + j] += temp * data->B[k * data->n + j];
            }
        }
    }
    pthread_exit(NULL);
}

// 2. 主线程：均分行数，分配任务创建线程
int chunk_size = m / num_threads;
for (int i = 0; i < num_threads; ++i) {
    // ...结构体传参省略...
```

```
thread_data[i].start_row = i * chunk_size;
thread_data[i].end_row = (i == num_threads - 1) ? m : (i + 1) * chunk_size;
pthread_create(&threads[i], NULL, gemm_worker, (void*)&thread_data[i]);
}

// 3. 收尾汇合
for (int i = 0; i < num_threads; ++i) {
    pthread_join(threads[i], NULL);
}
```

2.3 性能测试

测试基准：输入矩阵均为 2048 x 2048。

线程数	运行时间 (s)	加速比 (Sp)	并行效率 (Ep)
1	3.048	1.00	100%
2	1.702	1.79	89.5%
4	0.903	3.37	84.3%
8	0.549	5.55	69.4%
16	0.319	9.55	59.7%

2.4 性能分析

对于规整的密集型计算（如矩阵乘法），采用 Pthreads 可以获得相当出色的**强可扩展性 (Strong Scalability)**。在 1 到 16 线程范围内，随着线程的成倍增加，执行耗时成比例缩减。

3. 并行数组求和 (Pthreads)

3.1 算法设计与并行逻辑

与矩阵乘法的无冲突写入不同，数组求和的归约属于多对一计算，存在严重的写冲突可能。在本次实验中我们采取了如下**无锁设计与防伪共享 (False Sharing) 规避机制**：

- 1. **线程划分**：按数据位置均分数组成块。
- 2. **私有局部寄存器累加**：在子线程主循环里，使用当前线程私有局部变量 sum 累加该段数组的内容。如此既可以减少对主存的写入频次，也能使得缓存不被其他核心无效化。
- 3. **全局归约提取**：算毕，各子线程仅将唯一的 sum 修改至公有数组 local_sums[i]；最后主线程将不同位置的私有结果简单串行加总。

3.2 核心代码实现

数组求和的核心难度在于规避并发写时的缓存踩踏以及进行结果归约运算。

```
// 1. 无伪共享的安全局部求和（避免对 public 数组的频繁写入）
void* sum_worker(void* arg) {
    SumData* data = (SumData*)arg;
    long long sum = 0; // 私有寄存器/栈变量
    for (int i = data->start_idx; i < data->end_idx; ++i) {
```

```
        sum += data->A[i]; // 仅在 CPU 核心近端累加
    }
    *(data->local_sum) = sum; // 满载后，仅在循环结束写回一次主存
    pthread_exit(NULL);
}

// 2. 主线程的全局归约(Reduction)
long long total_sum = 0;
for (int i = 0; i < num_threads; ++i) {
    pthread_join(threads[i], NULL);
    // 等到子线程完成工作后，主线程安全地将各个坑位的结果相加
    total_sum += local_sums[i];
}
```

3.3 性能测试

测试分小规模 (1M) 和大规模 (128M) 两组：

1M 数据规模 (极小计算量)：

线程数	运行时间 (s)
1	0.000732
2	0.000365
4	0.000483
8	0.000409
16	0.000832

(注：当计算量极小时，创建和销毁 Pthreads 的系统开销掩盖了并行加速带来的收益，导致极小规模数据多线程反而变慢。)

128M 数据规模 (大计算量)：

线程数	运行时间 (s)	加速比 (Sp)
1	0.0322	1.00
2	0.0188	1.71
4	0.0136	2.36
8	0.0090	3.57
16	0.0079	4.07

3.4 性能分析与进阶探究：伪共享 (False Sharing) 的性能惩罚

在实现由于多线程间变量聚合而存在的竞争问题时，我们特地设计了两种不同的写法，以便在关闭编译器极致优化 (-O0) 的纯原生指令下，观察著名的缓存控制损耗现象——伪共享 (False Sharing)。

我们分别实现并测试了如下版本：

- 1. “**优化版本**”（使用线程专属寄存器累加）：通过私有局部变量 `sum` 进行累总，最后仅进行1次基于主存的写回。
- 2. “**伪共享反面教材**”（直接写回连续内存）：主循环内频繁对 `*(data->local_sum) += A[i]` 操作，导致相邻核心因同一缓存行(Cache Line) 被反复强刷，产生硬件级的读写踩踏。

纯净态 (-O0) 规模128M、16线程性能对比：

实现方式	耗时 (s)
私有寄存器 (无伪共享)	0.0160
频繁公存写 (伪共享)	0.0773

这证明，如果不注意对缓存行进行规避，不良的物理内存交互方式会在多核心间造成巨量的 L1 Cache Invalidation 惩罚风暴（伪共享版耗时是正常版的将近 5 倍）。